The Complete C++ Reference Report

The Complete C++ Reference Report

Every Core Term Explained with Examples

Table of Contents

---

1. Introduction to C++

C++ is a general-purpose, statically typed, compiled programming language created as an extension of C. It supports procedural, object-oriented, generic, and (since C++11) functional-style programming. It is designed for performance, giving programmers direct control over memory and hardware while also offering high-level abstractions such as classes, templates, and the Standard Template Library (STL).

C++ is used in operating systems, browsers, game engines, embedded systems, financial systems, and any domain where speed and control matter.

---

2. History and Evolution

Term: Bjarne Stroustrup — The Danish computer scientist who created C++ at Bell Labs, starting in 1979 as "C with Classes."

Term: ISO Standard — A formally ratified specification of the language. C++ has had standards in 1998, 2003, 2011, 2014, 2017, 2020, and 2023, each named after its year (e.g., "C++11," "C++20").

Example — evolution of a simple loop:

```
        // C++98 style
        for (std::vector<int>::iterator it = v.begin(); it != v.end(); ++it)
{
            std::cout << *it;
        }

        // C++11 and later (range-based for loop)
```

```cpp
    for (auto& x : v) {
        std::cout << x;
    }
```

This shows how the same task became simpler as the standard evolved.

---

3. Program Structure

Term: Header file — A file (commonly ending in `.h` or having no extension, e.g. `<iostream>`) containing declarations that are shared across source files.

Term: `#include` directive — Tells the preprocessor to insert the contents of a header file before compilation.

Term: `main()` function — The entry point of every C++ program; execution always starts here.

Term: Statement — A single instruction terminated by a semicolon `;`.

Term: Block — A group of statements enclosed in curly braces `{ }`.

Example:

```cpp
    #include <iostream>   // header inclusion

    int main() {           // entry point
        std::cout << "Hello, World!" << std::endl;  // statement
        return 0;           // exit code
    }                       // end of block
```

Term: `return` statement — Ends function execution and optionally returns a value to the caller. Returning `0` from `main()` conventionally signals successful execution.

---

4. Comments

Term: Single-line comment (`//`) — Everything after `//` on that line is ignored by the compiler.

Term: Multi-line comment (`/* */`) — Everything between `/*` and `*/` is ignored, even across multiple lines.

Example:

```cpp
    // This is a single-line comment
    int x = 5; // Explains that x holds a value of 5

    /* This is a
       multi-line comment
       spanning several lines */
```

Comments do not affect program execution; they exist purely for human readability and documentation.

---

5. Variables and Data Types

Term: Variable — A named storage location in memory that holds a value which can change during program execution.

Term: Declaration — Introducing a variable's name and type to the compiler (e.g., `int x;`).

Term: Initialization — Assigning an initial value at the time of declaration (e.g., `int x = 5;`).

Term: Data type — Specifies the kind of value a variable can hold and how much memory it occupies.

Fundamental Data Types

Type	Description	Typical Size	Example
`int`	Integer number	4 bytes	`int age = 25;`
`float`	Single-precision floating point	4 bytes	`float pi = 3.14f;`
`double`	Double-precision floating point	8 bytes	`double gravity = 9.81;`
`char`	Single character	1 byte	`char grade = 'A';`
`bool`	Boolean (true/false)	1 byte	`bool isValid = true;`
`void`	No value/type	—	used in function returns
`wchar_t`	Wide character	2 or 4 bytes	for Unicode characters

Term: `auto` keyword (C++11) — Lets the compiler deduce the variable's type automatically from its initializer.

Example:

```
        int age = 25;               // explicit int
        double price = 19.99;       // explicit double
        char initial = 'J';         // explicit char
        bool isReady = false;       // explicit bool

        auto count = 10;            // compiler deduces int
        auto name = "Claude";       // compiler deduces const char*
```

Term: Scope — The region of code in which a variable is visible and accessible. - **Local variable**: declared inside a function/block, accessible only within that block. - **Global variable**: declared outside all functions, accessible throughout the file.

Example:

```
        int globalCount = 0;   // global variable

        void increment() {
            int localStep = 1;  // local variable, only visible inside this
function
            globalCount += localStep;
        }
```

Term: Static variable — A variable declared with `static` that retains its value between function calls instead of being destroyed.

Example:

```
        void counter() {
            static int calls = 0; // initialized only once
```

```
        calls++;
        std::cout << calls << std::endl;
    }
    // Calling counter() three times prints 1, 2, 3
```

---

6. Constants and Literals

Term: Constant — A value that cannot be changed after initialization.

Term: `const` keyword — Declares a variable whose value is fixed at compile time or runtime but cannot be modified afterward.

Term: `constexpr` (C++11) — Declares a value that must be evaluable at compile time, enabling compiler optimizations.

Term: `#define` macro constant — A preprocessor-level constant substituted before compilation (older style, largely replaced by `const`/`constexpr`).

Example:

```
    const double PI = 3.14159;      // cannot be reassigned
    constexpr int MAX_SIZE = 100;   // evaluated at compile time
    #define GREETING "Hello"        // preprocessor macro
```

Term: Literal — A fixed value written directly in code. - Integer literal: `42` - Floating literal: `3.14` - Character literal: `'A'` - String literal: `"Hello"` - Boolean literal: `true` / `false`

Example:

```
    int x = 42;          // integer literal
    double y = 3.14;      // floating literal
    char c = 'A';         // character literal
    std::string s = "Hi"; // string literal
    bool flag = true;     // boolean literal
```

---

7. Operators

Term: Operator — A symbol that performs an operation on one or more operands.

Arithmetic Operators

`+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `%` (modulus/remainder)

```
    int a = 10, b = 3;
    std::cout << a + b; // 13
    std::cout << a % b; // 1 (remainder)
```

Relational Operators

`==`, `!=`, `>`, `<`, `>=`, `<=` — compare two values and return a `bool`.

```
    bool result = (5 > 3); // true
```

Logical Operators

`&&` (AND), `||` (OR), `!` (NOT)

```
        bool a = true, b = false;
        bool result = a && b; // false
```

Assignment Operators

=, +=, -=, *=, /=, %=

```
        int x = 5;
        x += 3; // x is now 8
```

Increment/Decrement Operators

++ (increment), -- (decrement); each has prefix and postfix forms.

```
        int i = 5;
        i++;   // post-increment: use i, then increase (i becomes 6)
        ++i;   // pre-increment: increase first, then use (i becomes 7)
```

Bitwise Operators

& (AND), | (OR), ^ (XOR), ~ (NOT), << (left shift), >> (right shift)

```
        int a = 5;   // 0101 in binary
        int b = 3;   // 0011 in binary
        int result = a & b; // 0001 -> 1
```

Ternary (Conditional) Operator

`condition ? expr1 : expr2`

```
        int a = 10, b = 20;
        int max = (a > b) ? a : b; // max = 20
```

---

8. Type Conversion & Casting

Term: Implicit conversion (coercion) — Automatic conversion performed by the compiler between compatible types.

```
        int i = 10;
        double d = i; // int implicitly converted to double: 10.0
```

Term: Explicit conversion (casting) — Manually converting one type to another.

Term: `static_cast` — Safe, compile-time-checked cast for related types (e.g., int to double).

```
        double d = 9.7;
        int i = static_cast<int>(d); // i = 9 (truncated)
```

Term: `dynamic_cast` — Runtime-checked cast, used with polymorphic classes (pointers/references) to safely downcast.

```
        Base* b = new Derived();
        Derived* d = dynamic_cast<Derived*>(b); // safe downcast
```

Term: `const_cast` — Adds or removes the `const` qualifier from a variable.

```
        const int x = 10;
        int& y = const_cast<int&>(x); // removes constness (use with
caution)
```

Term: `reinterpret_cast` — Reinterprets the bit pattern of one type as another; low-level and potentially unsafe.

```
        int i = 65;
        char* c = reinterpret_cast<char*>(&i);
```

---

9. Control Structures

Term: Conditional statement — Executes code only if a condition is true.

`if / else if / else`

```
        int score = 85;
        if (score >= 90) {
            std::cout << "Grade A";
        } else if (score >= 80) {
            std::cout << "Grade B";
        } else {
            std::cout << "Grade C or below";
        }
```

`switch` statement

Term: `switch` — Selects one of several code blocks to execute based on the value of an expression.

```
        int day = 3;
        switch (day) {
            case 1: std::cout << "Monday"; break;
            case 2: std::cout << "Tuesday"; break;
            case 3: std::cout << "Wednesday"; break;
            default: std::cout << "Unknown day";
        }
```

Term: `break` — Exits the current loop or switch block immediately.
Term: `default` — The fallback case in a switch statement when no other case matches.

Loops

Term: `for` loop — Repeats a block a specific number of times, controlled by an initializer, condition, and increment.

```
        for (int i = 0; i < 5; i++) {
            std::cout << i << " ";
        }
        // Output: 0 1 2 3 4
```

Term: `while` loop — Repeats a block as long as a condition remains true, checked before each iteration.

```
        int i = 0;
        while (i < 5) {
            std::cout << i << " ";
            i++;
        }
```

Term: `do-while` loop — Like `while`, but the condition is checked after the block executes, guaranteeing at least one execution.

```
        int i = 0;
        do {
```

```
        std::cout << i << " ";
        i++;
    } while (i < 5);
```

Term: Range-based `for` loop (C++11) — Iterates directly over elements of a container.

```
    std::vector<int> nums = {1, 2, 3};
    for (int n : nums) {
        std::cout << n << " ";
    }
```

Term: `continue` — Skips the rest of the current loop iteration and moves to the next one.

```
    for (int i = 0; i < 5; i++) {
        if (i == 2) continue; // skips printing 2
        std::cout << i << " ";
    }
    // Output: 0 1 3 4
```

---

10. Functions

Term: Function — A named, reusable block of code that performs a specific task and can accept inputs (parameters) and return a value.

Term: Function signature — The combination of a function's name and parameter types, which distinguishes it from other functions.

Term: Parameter vs. Argument — A parameter is the variable listed in a function's definition; an argument is the actual value passed when calling the function.

```
    // Function declaration (prototype)
    int add(int a, int b);

    // Function definition
    int add(int a, int b) {   // a, b are parameters
        return a + b;
    }

    int main() {
        int result = add(3, 4); // 3, 4 are arguments
        std::cout << result;    // 7
    }
```

Term: Function overloading — Defining multiple functions with the same name but different parameter lists.

```
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
```

Term: Default argument — A parameter value used automatically if the caller doesn't supply one.

```
    void greet(std::string name = "Guest") {
        std::cout << "Hello, " << name;
    }
    greet();          // Hello, Guest
    greet("Claude");  // Hello, Claude
```

Term: Pass by value — The function receives a copy of the argument; changes inside the function don't affect the original.

```
    void increment(int x) { x++; }  // original variable unaffected
```

Term: Pass by reference — The function receives a reference to the original variable; changes inside the function do affect the original.

```
void increment(int& x) { x++; } // original variable is modified
```

Term: Recursion — A function that calls itself to solve a smaller instance of the same problem.

```
int factorial(int n) {
    if (n <= 1) return 1;       // base case
    return n * factorial(n - 1); // recursive case
}
```

Term: Inline function — A function marked with `inline`, suggesting the compiler substitute the function's code directly at the call site to avoid call overhead.

```
inline int square(int x) { return x * x; }
```

---

11. Arrays

Term: Array — A fixed-size collection of elements of the same type, stored in contiguous memory.

Term: Index — The position of an element in an array, starting at 0.

```
int numbers[5] = {10, 20, 30, 40, 50};
std::cout << numbers[0]; // 10 (first element)
std::cout << numbers[4]; // 50 (last element)
```

Term: Multidimensional array — An array of arrays, commonly used to represent grids or matrices.

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
std::cout << matrix[1][2]; // 6
```

Term: `std::array` (C++11) — A fixed-size array container from the STL that is safer and offers more functionality than a raw array.

```
#include <array>
std::array<int, 3> arr = {1, 2, 3};
std::cout << arr.size(); // 3
```

Term: Array decay — When an array is passed to a function, it "decays" into a pointer to its first element, losing size information.

```
void printArray(int* arr, int size) {
    for (int i = 0; i < size; i++) std::cout << arr[i] << " ";
}
```

---

12. Strings

Term: C-style string — An array of characters terminated by a null character `'\0'`.

```
char greeting[] = "Hello"; // stored as 'H','e','l','l','o','\0'
```

Term: `std::string` — The C++ Standard Library class representing a dynamically-sized, safer string type.

```cpp
#include <string>
std::string name = "Claude";
name += " Sonnet";               // concatenation
std::cout << name.length();      // 13
std::cout << name.substr(0, 6);  // "Claude"
```

Term: String concatenation — Joining two or more strings together, typically with `+` or `+=`.

```cpp
std::string first = "Hello, ";
std::string second = "World!";
std::string combined = first + second; // "Hello, World!"
```

Term: String comparison — Comparing strings lexicographically using `==`, `<`, `>`, etc.

```cpp
std::string a = "apple", b = "banana";
bool result = (a < b); // true, 'a' < 'b' alphabetically
```

Term: String stream (`std::stringstream`) — Allows treating strings like input/output streams, useful for parsing or building strings.

```cpp
#include <sstream>
std::stringstream ss;
ss << "Value: " << 42;
std::string result = ss.str(); // "Value: 42"
```

---

13. Pointers

Term: Pointer — A variable that stores the memory address of another variable.

Term: Dereference operator (`*`) — Accesses the value stored at the address a pointer points to.

Term: Address-of operator (`&`) — Retrieves the memory address of a variable.

```cpp
int x = 10;
int* ptr = &x;       // ptr holds the address of x
std::cout << *ptr;    // dereference: prints 10
*ptr = 20;            // modifies x through the pointer
std::cout << x;       // 20
```

Term: Null pointer — A pointer that points to no valid memory location. In modern C++, represented by `nullptr` (C++11).

```cpp
int* p = nullptr;
if (p == nullptr) {
    std::cout << "Pointer is null";
}
```

Term: Dangling pointer — A pointer that references memory that has already been freed or gone out of scope; using it causes undefined behavior.

```cpp
int* danger;
{
    int temp = 5;
    danger = &temp;
} // temp goes out of scope here
// danger now dangles — using *danger is undefined behavior
```

Term: Pointer arithmetic — Performing addition/subtraction on pointers to move between memory addresses, typically used with arrays.

```
int arr[3] = {10, 20, 30};
int* p = arr;
std::cout << *(p + 1); // 20 (second element)
```

Term: Void pointer (`void*`) — A generic pointer that can point to any data type but must be cast before dereferencing.

```
void* vp;
int x = 5;
vp = &x;
std::cout << *static_cast<int*>(vp); // 5
```

Term: Pointer to pointer — A pointer that stores the address of another pointer.

```
int x = 10;
int* p = &x;
int** pp = &p;
std::cout << **pp; // 10
```

---

14. References

Term: Reference — An alias for an existing variable; once bound, it always refers to that same variable and cannot be null or reseated.

```
int x = 10;
int& ref = x;  // ref is now an alias for x
ref = 20;      // modifies x directly
std::cout << x; // 20
```

Term: Reference vs. pointer — References must be initialized when declared and cannot be changed to refer to a different variable; pointers can be reassigned and can be null.

Term: `const` reference — A reference that cannot be used to modify the value it refers to; commonly used for efficient, read-only function parameters.

```
void printValue(const std::string& text) {
    std::cout << text; // text cannot be modified here
}
```

Term: Rvalue reference (`&&`, C++11) — A reference that binds to temporary (rvalue) objects, enabling move semantics.

```
void process(std::string&& temp) {
    std::cout << "Processing temporary: " << temp;
}
process("Hello"); // binds to a temporary string
```

---

15. Dynamic Memory Management

Term: Dynamic memory allocation — Allocating memory at runtime (on the “heap”) rather than at compile time.

Term: `new` operator — Allocates memory on the heap and returns a pointer to it.

Term: `delete` operator — Frees memory previously allocated with `new`, returning it to the system.

```
        int* ptr = new int;      // allocate a single integer
        *ptr = 42;
        std::cout << *ptr;       // 42
        delete ptr;               // free the memory
        ptr = nullptr;             // good practice: avoid dangling pointer

        int* arr = new int[5];    // allocate an array of 5 integers
        delete[] arr;              // free array memory (note the [])
```

Term: Memory leak — Memory allocated with `new` that is never freed with `delete`, causing the program to consume increasing amounts of memory over time.

```
        void leak() {
            int* p = new int(5);
            // missing delete p; -> memory leak every time this function
runs
        }
```

Term: Heap vs. Stack — - **Stack**: automatically managed memory for local variables; fast, but limited in size and freed automatically when a function returns. - **Heap**: manually managed memory for dynamic allocation; larger, but must be explicitly freed or managed with smart pointers.

Term: Double free — Calling `delete` on the same pointer twice, which causes undefined behavior.

Term: RAII (Resource Acquisition Is Initialization) — A C++ idiom where resource management (memory, files, locks) is tied to object lifetime: resources are acquired in a constructor and released in a destructor.

```
        class FileHandler {
            FILE* file;
        public:
            FileHandler(const char* name) { file = fopen(name, "r"); } //
acquire
            ~FileHandler() { if (file) fclose(file); }                //
release
        };
        // When a FileHandler object goes out of scope, the file is
automatically closed.
```

---

16. Structures and Unions

Term: `struct` — A user-defined type that groups related variables (members) together, public by default.

```
        struct Point {
            int x;
            int y;
        };

        Point p1;
        p1.x = 10;
        p1.y = 20;
        std::cout << p1.x << ", " << p1.y; // 10, 20
```

Term: `union` — A user-defined type similar to `struct`, but all members share the same memory location, so only one member holds a valid value at a time.

```cpp
union Data {
    int i;
    float f;
};

Data d;
d.i = 10;
std::cout << d.i;  // 10
d.f = 3.14;         // overwrites the same memory as d.i
std::cout << d.f;   // 3.14 (d.i is no longer valid)
```

Term: `struct` vs. `class` — Functionally similar in C++; the only difference is default access: `struct` members are `public` by default, `class` members are `private` by default.

---

17. Enumerations

Term: `enum` — A user-defined type consisting of a set of named integer constants.

```cpp
enum Color { RED, GREEN, BLUE }; // RED=0, GREEN=1, BLUE=2 by default
Color favorite = GREEN;
if (favorite == GREEN) {
    std::cout << "Favorite color is green";
}
```

Term: `enum class` (scoped enum, C++11) — A safer version of `enum` where values are scoped to the enum's name and don't implicitly convert to integers, preventing naming collisions.

```cpp
enum class TrafficLight { Red, Yellow, Green };
TrafficLight light = TrafficLight::Green;
if (light == TrafficLight::Green) {
    std::cout << "Go!";
}
```

---

18. Object-Oriented Programming

Term: Class — A user-defined blueprint for creating objects, bundling data (member variables) and behavior (member functions) together.

Term: Object — A concrete instance of a class.

```cpp
class Car {
public:
    std::string brand;
    int speed;

    void accelerate() {
        speed += 10;
    }
};

int main() {
    Car myCar;           // myCar is an object of class Car
    myCar.brand = "Toyota";
    myCar.speed = 0;
    myCar.accelerate();
    std::cout << myCar.speed; // 10
}
```

Term: Member variable (attribute/field) — A variable that belongs to a class and stores an object's state (e.g., `brand`, `speed` above).

Term: Member function (method) — A function that belongs to a class and defines an object's behavior (e.g., `accelerate()` above).

Term: Access specifier — Keywords controlling visibility of class members: - **`public`** — accessible from anywhere the object is visible. - **`private`** — accessible only within the class itself. - **`protected`** — accessible within the class and its derived classes.

```
class BankAccount {
private:
    double balance; // hidden from outside access

public:
    void deposit(double amount) {
        balance += amount;
    }
    double getBalance() {
        return balance;
    }
};
```

Term: `this` pointer — An implicit pointer available inside every non-static member function, pointing to the object that invoked the function.

```
class Point {
    int x;
public:
    void setX(int x) {
        this->x = x; // disambiguates member x from parameter x
    }
};
```

Term: Static member — A class member shared by all objects of the class rather than belonging to any single instance.

```
class Counter {
public:
    static int count; // declared here
};
int Counter::count = 0; // defined outside the class

// Usage: Counter::count accesses the shared variable directly
```

---

19. Constructors and Destructors

Term: Constructor — A special member function automatically called when an object is created, typically used to initialize member variables. It has the same name as the class and no return type.

```
class Person {
public:
    std::string name;
    Person(std::string n) {   // constructor
        name = n;
        std::cout << "Person created: " << name;
    }
};
Person p("Alice"); // constructor runs automatically
```

Term: Default constructor — A constructor with no parameters (or all default parameters); it's what runs if you write `ClassName obj;`.

Term: Parameterized constructor — A constructor that accepts arguments to initialize members with custom values (as shown above).

Term: Copy constructor — A constructor that initializes an object using another object of the same class, copying its member values.

```
class Point {
public:
    int x, y;
    Point(int a, int b) : x(a), y(b) {}
    Point(const Point& other) {  // copy constructor
        x = other.x;
        y = other.y;
    }
};
Point p1(1, 2);
Point p2 = p1; // copy constructor invoked
```

Term: Member initializer list — A syntax (`: x(a), y(b)`) used to initialize member variables directly, more efficient than assigning inside the constructor body.

Term: Destructor — A special member function automatically called when an object is destroyed (e.g., goes out of scope or is deleted), used to release resources. Denoted by `~ClassName()`.

```
class Person {
public:
    ~Person() {  // destructor
        std::cout << "Person destroyed";
    }
};
// Destructor runs automatically when the object's lifetime ends
```

Term: Constructor overloading — Defining multiple constructors with different parameter lists, similar to function overloading.

---

20. Inheritance

Term: Inheritance — A mechanism where a new class (derived/child class) acquires properties and behaviors from an existing class (base/parent class).

```
class Animal {           // base class
public:
    void eat() {
        std::cout << "This animal eats food.";
    }
};

class Dog : public Animal {  // derived class
public:
    void bark() {
        std::cout << "The dog barks.";
    }
};

int main() {
    Dog myDog;
    myDog.eat();  // inherited from Animal
    myDog.bark(); // defined in Dog
}
```

Term: Base class (parent/superclass) — The class being inherited from (`Animal` above).

Term: Derived class (child/subclass) — The class that inherits from another (`Dog` above).

Term: Inheritance access modes — Determine how inherited members' access levels change in the derived class: - `public` inheritance: public/protected members keep their access level. - `protected` inheritance: public/protected members become protected. - `private` inheritance: public/protected members become private.

Term: Single inheritance — A derived class inherits from exactly one base class (as shown above).

Term: Multiple inheritance — A derived class inherits from more than one base class.

```
        class Flyable { public: void fly() { std::cout << "Flying"; } };
        class Swimmable { public: void swim() { std::cout << "Swimming"; }
};
        class Duck : public Flyable, public Swimmable {};
```

Term: Multilevel inheritance — A chain of inheritance where a derived class becomes the base class for another.

```
        class Animal {};
        class Mammal : public Animal {};
        class Dog : public Mammal {};
```

Term: Function overriding — A derived class providing its own implementation of a function already defined in its base class, with the same signature.

```
        class Animal {
        public:
            virtual void sound() { std::cout << "Some sound"; }
        };
        class Cat : public Animal {
        public:
            void sound() override { std::cout << "Meow"; } // overrides base
version
        };
```

---

21. Polymorphism

Term: Polymorphism — The ability of different object types to be treated through a common interface, with behavior determined by the actual object type at runtime or compile time.

Term: Compile-time polymorphism (static) — Achieved through function overloading and operator overloading; resolved during compilation.

Term: Runtime polymorphism (dynamic) — Achieved through virtual functions and inheritance; resolved during program execution.

Term: Virtual function — A member function declared with `virtual` in a base class, which can be overridden in derived classes; enables dynamic dispatch.

```
        class Shape {
        public:
            virtual double area() {  // virtual function
                return 0;
            }
        };
```

```cpp
class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area() override {  // overridden
        return 3.14159 * radius * radius;
    }
};

int main() {
    Shape* shape = new Circle(5);
    std::cout << shape->area(); // calls Circle's area() at runtime:
78.54
    delete shape;
}
```

Term: Pure virtual function — A virtual function with no implementation, declared with `= 0`, forcing derived classes to provide their own implementation.

Term: Abstract class — A class containing at least one pure virtual function; it cannot be instantiated directly and serves as an interface/base for derived classes.

```cpp
class Shape {
public:
    virtual double area() = 0; // pure virtual function
};
// Shape myShape; // ERROR: cannot instantiate an abstract class

class Square : public Shape {
    double side;
public:
    Square(double s) : side(s) {}
    double area() override { return side * side; }
};
```

Term: Virtual destructor — A destructor declared `virtual` in a base class to ensure the correct derived class destructor runs when deleting through a base class pointer.

```cpp
class Base {
public:
    virtual ~Base() { std::cout << "Base destroyed"; }
};
class Derived : public Base {
public:
    ~Derived() { std::cout << "Derived destroyed"; }
};
Base* b = new Derived();
delete b; // prints "Derived destroyed" then "Base destroyed"
```

---

22. Encapsulation and Abstraction

Term: Encapsulation — Bundling data and the methods that operate on it within a single unit (class), while restricting direct access to internal state using access specifiers (`private`/`protected`).

```cpp
class BankAccount {
private:
    double balance; // hidden internal state

public:
    void deposit(double amount) {
```

```
            if (amount > 0) balance += amount; // controlled access
        }
        double getBalance() const {
            return balance;
        }
    };
    // Outside code cannot do account.balance = -1000; directly
```

Term: Getter/Setter — Public methods that provide controlled read (`getBalance`) and write (`deposit`/`setBalance`) access to private data.

Term: Abstraction — Hiding complex implementation details and exposing only the essential features of an object, typically through abstract classes or interfaces.

```
    class Vehicle {
    public:
        virtual void start() = 0; // interface; details hidden from the
user
    };
    class Car : public Vehicle {
    public:
        void start() override {
            std::cout << "Turning key, engine starts."; //
implementation hidden from caller
        }
    };
```

---

23. Operator Overloading

Term: Operator overloading — Redefining how an existing operator (+, ==, <<, etc.) behaves when used with user-defined types.

```
    class Complex {
    public:
        double real, imag;
        Complex(double r, double i) : real(r), imag(i) {}

        Complex operator+(const Complex& other) {  // overload +
            return Complex(real + other.real, imag + other.imag);
        }
    };

    int main() {
        Complex a(1, 2), b(3, 4);
        Complex c = a + b; // uses overloaded operator+
        std::cout << c.real << ", " << c.imag; // 4, 6
    }
```

Term: Friend function — A non-member function granted access to a class's private members, often used to overload operators like << for printing.

```
    class Point {
        int x, y;
    public:
        Point(int a, int b) : x(a), y(b) {}
        friend std::ostream& operator<<(std::ostream& os, const Point&
p);
    };
    std::ostream& operator<<(std::ostream& os, const Point& p) {
        os << "(" << p.x << ", " << p.y << ")";
        return os;
    }
```

```
        // Usage: std::cout << myPoint; prints "(1, 2)"
```

---

24. Templates and Generic Programming

Term: Template — A blueprint for creating generic functions or classes that work with any data type, determined when the template is used.

Term: Function template

```
        template <typename T>
        T maxValue(T a, T b) {
            return (a > b) ? a : b;
        }

        int main() {
            std::cout << maxValue(3, 7);      // works with int: 7
            std::cout << maxValue(2.5, 1.2);   // works with double: 2.5
        }
```

Term: Class template

```
        template <typename T>
        class Box {
            T value;
        public:
            Box(T v) : value(v) {}
            T getValue() { return value; }
        };

        int main() {
            Box<int> intBox(10);
            Box<std::string> strBox("Hello");
            std::cout << intBox.getValue();  // 10
            std::cout << strBox.getValue();  // Hello
        }
```

Term: Template specialization — Providing a custom implementation of a template for a specific type.

```
        template <typename T>
        class Printer {
        public:
            void print(T value) { std::cout << value; }
        };

        template <>  // full specialization for bool
        class Printer<bool> {
        public:
            void print(bool value) { std::cout << (value ? "true" :
"false"); }
        };
```

Term: Variadic template (C++11) — A template that accepts a variable number of arguments of any type.

```
        template<typename T>
        T sum(T value) {
            return value;
        }
        template<typename T, typename... Args>
        T sum(T first, Args... rest) {
            return first + sum(rest...);
        }
        // sum(1, 2, 3, 4) returns 10
```

Term: Concept (C++20) — A named constraint used to restrict what types a template parameter may accept, improving error messages and readability.

```cpp
#include <concepts>
template <typename T>
concept Numeric = std::is_arithmetic_v<T>;

template <Numeric T>
T add(T a, T b) { return a + b; }
```

---

25. The Standard Template Library (STL)

Term: STL — A collection of generic, reusable template-based classes and functions providing common data structures and algorithms.

Containers

Term: `std::vector` — A dynamic array that can grow or shrink in size automatically.

```cpp
#include <vector>
std::vector<int> nums = {1, 2, 3};
nums.push_back(4);      // adds 4 to the end
std::cout << nums[3];    // 4
std::cout << nums.size(); // 4
```

Term: `std::list` — A doubly linked list, allowing efficient insertion/removal at any position.

```cpp
#include <list>
std::list<int> l = {1, 2, 3};
l.push_front(0); // adds 0 to the beginning
```

Term: `std::map` — An associative container storing key-value pairs, sorted by key.

```cpp
#include <map>
std::map<std::string, int> ages;
ages["Alice"] = 30;
ages["Bob"] = 25;
std::cout << ages["Alice"]; // 30
```

Term: `std::unordered_map` — Like `std::map` but stored using a hash table, offering average O(1) lookup without ordering.

```cpp
#include <unordered_map>
std::unordered_map<std::string, int> scores;
scores["Alice"] = 95;
```

Term: `std::set` — A container of unique, sorted elements.

```cpp
#include <set>
std::set<int> s = {3, 1, 2, 1}; // duplicates ignored
// s contains {1, 2, 3} in sorted order
```

Term: `std::stack` — A LIFO (Last-In-First-Out) container adapter.

```cpp
#include <stack>
std::stack<int> st;
st.push(1);
st.push(2);
std::cout << st.top(); // 2
st.pop();              // removes 2
```

Term: `std::queue` — A FIFO (First-In-First-Out) container adapter.

```
        #include <queue>
        std::queue<int> q;
        q.push(1);
        q.push(2);
        std::cout << q.front(); // 1
        q.pop();                 // removes 1
```

Iterators

Term: Iterator — An object that points to an element within a container and can be used to traverse it, similar in concept to a pointer.

```
        std::vector<int> v = {10, 20, 30};
        for (std::vector<int>::iterator it = v.begin(); it != v.end(); ++it)
{
            std::cout << *it << " "; // 10 20 30
        }
```

Term: `begin()` / `end()` — Return iterators pointing to the first element and to "one past the last" element, respectively.

Algorithms

Term: `std::sort` — Sorts a range of elements in ascending order by default.

```
        #include <algorithm>
        std::vector<int> v = {5, 2, 8, 1};
        std::sort(v.begin(), v.end()); // v becomes {1, 2, 5, 8}
```

Term: `std::find` — Searches a range for the first occurrence of a value, returning an iterator to it (or `end()` if not found).

```
        auto it = std::find(v.begin(), v.end(), 8);
        if (it != v.end()) std::cout << "Found!";
```

Term: `std::transform` — Applies a function to each element of a range, storing results in another range.

```
        std::vector<int> src = {1, 2, 3};
        std::vector<int> dst(3);
        std::transform(src.begin(), src.end(), dst.begin(), [](int x) {
return x * 2; });
        // dst becomes {2, 4, 6}
```

Term: `std::accumulate` — Sums (or otherwise combines) all elements in a range into a single value.

```
        #include <numeric>
        std::vector<int> v = {1, 2, 3, 4};
        int total = std::accumulate(v.begin(), v.end(), 0); // 10
```

Term: Pair (`std::pair`) — Holds two heterogeneous values together.

```
        #include <utility>
        std::pair<std::string, int> person("Alice", 30);
        std::cout << person.first << ", " << person.second; // Alice, 30
```

Term: Tuple (`std::tuple`, C++11) — Like `std::pair` but can hold any number of heterogeneous values.

```
        #include <tuple>
        std::tuple<std::string, int, double> record("Bob", 25, 5.9);
        std::cout << std::get<0>(record); // "Bob"
```

26. Exception Handling

Term: Exception — An abnormal event or error condition that disrupts normal program flow, which can be "thrown" and "caught."

Term: `try` block — Encloses code that might throw an exception.

Term: `throw` statement — Signals that an exception has occurred, passing an object describing the error.

Term: `catch` block — Handles an exception thrown from a corresponding `try` block.

```
        #include <stdexcept>

        double divide(double a, double b) {
            if (b == 0) {
                throw std::runtime_error("Division by zero!"); // throw
exception
            }
            return a / b;
        }

        int main() {
            try {
                std::cout << divide(10, 0);
            } catch (const std::runtime_error& e) {  // catch specific
exception
                std::cout << "Error: " << e.what();
            } catch (...) {                          // catch-all handler
                std::cout << "Unknown error occurred";
            }
        }
```

Term: `std::exception` — The base class for all standard exception types (e.g., `std::runtime_error`, `std::logic_error`, `std::out_of_range`).

Term: `finally`-equivalent (no direct keyword) — C++ has no `finally` block; RAII (destructors) is used instead to guarantee cleanup regardless of exceptions.

Term: Custom exception class — A user-defined class, typically derived from `std::exception`, to represent domain-specific errors.

```
        class InsufficientFundsException : public std::exception {
        public:
            const char* what() const noexcept override {
                return "Insufficient funds for this transaction.";
            }
        };
```

Term: Exception safety — The guarantee a piece of code makes about program state if an exception occurs (basic, strong, or no-throw guarantee).

---

27. File Handling (I/O Streams)

Term: Stream — An abstraction representing a sequence of data flowing to or from a source (file, console, etc.).

Term: `std::ifstream` — Input file stream, used for reading from files.

Term: `std::ofstream` — Output file stream, used for writing to files.

Term: `std::fstream` — A stream that supports both reading and writing to files.

```
        #include <fstream>
        #include <string>

        int main() {
            // Writing to a file
            std::ofstream outFile("data.txt");
            if (outFile.is_open()) {
                outFile << "Hello, file!" << std::endl;
                outFile.close();
            }

            // Reading from a file
            std::ifstream inFile("data.txt");
            std::string line;
            if (inFile.is_open()) {
                while (std::getline(inFile, line)) {
                    std::cout << line << std::endl;
                }
                inFile.close();
            }
        }
```

Term: `std::getline` — Reads an entire line of text from a stream into a string, up to a newline character.

Term: File modes — Flags controlling how a file is opened: `std::ios::in` (read), `std::ios::out` (write), `std::ios::app` (append), `std::ios::binary` (binary mode).

```
        std::ofstream logFile("log.txt", std::ios::app); // append mode
        logFile << "New log entry\n";
```

---

28. Namespaces

Term: Namespace — A declarative region that groups related names (classes, functions, variables) together to avoid naming conflicts.

```
        namespace MathUtils {
            int square(int x) {
                return x * x;
            }
        }

        int main() {
            std::cout << MathUtils::square(5); // 25, accessed via scope
resolution
        }
```

Term: `using` directive — Brings names from a namespace into the current scope, avoiding the need to qualify them.

```
        using namespace std; // brings all std:: names into scope
        cout << "No need for std:: prefix now" << endl;
```

Term: Scope resolution operator (`::`) — Used to specify which namespace, class, or global scope a name belongs to.

```
        int x = 10;         // global x
        void foo() {
            int x = 20;      // local x
```

```
            std::cout << ::x; // accesses global x using scope resolution: 10
        }
```

---

29. Preprocessor Directives

Term: Preprocessor — A program that processes source code before actual compilation, handling directives that begin with `#`.

Term: `#include` — Inserts the content of another file (usually a header) into the current file.

Term: `#define` — Defines a macro: a piece of text substituted wherever the macro name appears.

```
        #define PI 3.14159
        #define SQUARE(x) ((x) * (x))
        std::cout << SQUARE(5); // expands to ((5) * (5)) = 25
```

Term: Conditional compilation (`#ifdef`, `#ifndef`, `#endif`) — Includes or excludes code based on whether a macro is defined; commonly used for header guards.

```
        #ifndef MYHEADER_H   // if MYHEADER_H is not yet defined
        #define MYHEADER_H    // define it now

        // header content goes here

        #endif                // end of conditional block
```

Term: Header guard — The `#ifndef`/`#define`/`#endif` pattern above, preventing a header file from being included multiple times in the same translation unit.

Term: `#pragma once` — A non-standard but widely supported alternative to header guards, telling the compiler to include the file only once.

---

30. Smart Pointers and Modern Memory Management

Term: Smart pointer — A class that wraps a raw pointer and automatically manages the memory it points to, following RAII principles to prevent leaks.

Term: `std::unique_ptr` (C++11) — Owns a dynamically allocated object exclusively; only one `unique_ptr` can own a given object at a time, and it's automatically deleted when the pointer goes out of scope.

```
        #include <memory>
        std::unique_ptr<int> ptr = std::make_unique<int>(42);
        std::cout << *ptr; // 42
        // Memory automatically freed when ptr goes out of scope; cannot be copied
```

Term: `std::shared_ptr` (C++11) — Allows multiple pointers to share ownership of the same object, using reference counting; the object is deleted when the last `shared_ptr` referencing it is destroyed.

```
        std::shared_ptr<int> p1 = std::make_shared<int>(10);
        std::shared_ptr<int> p2 = p1;    // both share ownership
```

```
        std::cout << p1.use_count();      // 2
```

Term: `std::weak_ptr` (C++11) — A non-owning reference to an object managed by a `shared_ptr`, used to break circular reference cycles.

```
        std::shared_ptr<int> sp = std::make_shared<int>(5);
        std::weak_ptr<int> wp = sp; // does not increase reference count
        if (auto locked = wp.lock()) {  // safely access if still alive
            std::cout << *locked;
        }
```

Term: Move semantics (C++11) — A mechanism to transfer ownership of resources from one object to another without copying, using rvalue references.

Term: `std::move` — Casts an object to an rvalue reference, signaling that its resources can be "moved" rather than copied.

```
        #include <utility>
        std::vector<int> a = {1, 2, 3};
        std::vector<int> b = std::move(a); // ownership transferred to b
        // a is now left in a valid but unspecified (typically empty) state
```

Term: Move constructor / move assignment operator — Special member functions that implement move semantics for a user-defined class.

```
        class Buffer {
            int* data;
        public:
            Buffer(Buffer&& other) noexcept {  // move constructor
                data = other.data;
                other.data = nullptr;
            }
        };
```

---

31. Lambda Expressions

Term: Lambda expression (C++11) — An anonymous, inline function that can be defined and used directly where needed, often for short callbacks passed to algorithms.

```
        auto add = [](int a, int b) {  // lambda stored in variable "add"
            return a + b;
        };
        std::cout << add(3, 4); // 7
```

Term: Capture list (`[]`) — Specifies which surrounding variables a lambda can access and how (by value or by reference).

```
        int x = 10;
        auto byValue = [x]() { std::cout << x; };       // captures x by
value (copy)
        auto byRef = [&x]() { x++; };                    // captures x by
reference
        auto captureAll = [=]() { std::cout << x; };     // captures all
used variables by value
        auto captureAllRef = [&]() { x++; };             // captures all
used variables by reference
```

Term: Lambda with STL algorithms — Lambdas are frequently passed as predicates or operations to STL algorithms.

```
        std::vector<int> nums = {1, 2, 3, 4, 5};
        int count = std::count_if(nums.begin(), nums.end(), [](int n) {
```

```
        return n % 2 == 0; // predicate: is n even?
    });
    std::cout << count; // 2 (2 and 4 are even)
```

---

32. Multithreading and Concurrency

Term: Thread — An independent path of execution that can run concurrently with other threads within the same program.

Term: `std::thread` (C++11) — The standard class for creating and managing threads.

```
    #include <thread>

    void printMessage() {
        std::cout << "Hello from thread!";
    }

    int main() {
        std::thread t(printMessage); // starts a new thread
        t.join();                    // waits for the thread to finish
    }
```

Term: `join()` — Blocks the calling thread until the specified thread finishes execution.

Term: `detach()` — Separates a thread from the thread object, allowing it to run independently in the background.

Term: Race condition — A bug that occurs when multiple threads access shared data simultaneously, and the outcome depends on unpredictable timing.

Term: `std::mutex` — A mutual exclusion lock that ensures only one thread can access a shared resource at a time, preventing race conditions.

```
    #include <mutex>
    std::mutex mtx;
    int sharedCounter = 0;

    void increment() {
        mtx.lock();
        sharedCounter++;
        mtx.unlock();
    }
```

Term: `std::lock_guard` — A RAII wrapper around a mutex that automatically locks it on construction and unlocks it on destruction, even if an exception occurs.

```
    void increment() {
        std::lock_guard<std::mutex> lock(mtx); // locked here
        sharedCounter++;
    } // automatically unlocked when lock goes out of scope
```

Term: `std::atomic` — A template providing lock-free, thread-safe operations on simple data types.

```
    #include <atomic>
    std::atomic<int> counter(0);
    counter++; // thread-safe increment without an explicit mutex
```

Term: Deadlock — A situation where two or more threads are each waiting for the other to release a resource, causing the program to freeze indefinitely.

---

33. Modern C++ Features (C++11–C++23)

Term: `nullptr` (C++11) — A type-safe null pointer literal, replacing the old `NULL` macro.

Term: `auto` (C++11) — Automatic type deduction (see Section 5).

Term: Range-based for loop (C++11) — Simplified iteration syntax (see Section 9).

Term: Structured bindings (C++17) — Allows unpacking multiple values (e.g., from a pair, tuple, or struct) into named variables in one statement.

```
        std::pair<int, std::string> data = {1, "Alice"};
        auto [id, name] = data; // id = 1, name = "Alice"
```

Term: `if constexpr` (C++17) — A compile-time conditional that discards the branch not taken, useful in template code.

```
        template <typename T>
        void process(T value) {
            if constexpr (std::is_integral<T>::value) {
                std::cout << "Integer type";
            } else {
                std::cout << "Non-integer type";
            }
        }
```

Term: `std::optional` (C++17) — Represents a value that may or may not be present, avoiding the need for sentinel values like `-1` or null pointers.

```
        #include <optional>
        std::optional<int> findValue(bool found) {
            if (found) return 42;
            return std::nullopt; // no value
        }
```

Term: `std::variant` (C++17) — A type-safe union that can hold a value of one of several specified types.

```
        #include <variant>
        std::variant<int, std::string> data = "Hello";
        data = 42; // now holds an int
```

Term: Concepts (C++20) — Named constraints on template parameters (see Section 24).

Term: Ranges library (C++20) — Provides a more expressive, composable way to work with sequences of elements, using “views” and pipes.

```
        #include <ranges>
        std::vector<int> nums = {1, 2, 3, 4, 5};
        auto even = nums | std::views::filter([](int n) { return n % 2 == 0;
});
```

Term: Coroutines (C++20) — Functions that can suspend execution and resume later, useful for asynchronous programming, using `co_await`, `co_yield`, `co_return`.

Term: Modules (C++20) — A new mechanism replacing traditional headers, improving compile times and encapsulation using the `import`/`export` keywords.

Term: `std::format` (C++20) — A type-safe, Python-like string formatting function.

```
        #include <format>
        std::string s = std::format("{} is {} years old", "Alice", 30);
```

---

34. Glossary of Additional Terms

Compiler — A program that translates C++ source code into machine code (e.g., GCC, Clang, MSVC).

Linker — A tool that combines compiled object files and libraries into a single executable.

Undefined behavior (UB) — Code whose behavior the C++ standard does not define, meaning anything could happen (crash, wrong output, or appear to work); common causes include dereferencing null/dangling pointers and out-of-bounds array access.

Object lifetime — The period between an object's construction and destruction.

Compilation unit (translation unit) — A single source file after preprocessing, ready to be compiled independently.

Header-only library — A library implemented entirely within header files, requiring no separate compilation/linking step.

ABI (Application Binary Interface) — The low-level interface between compiled binary components, including calling conventions and data layout.

Undefined vs. Implementation-defined behavior — Undefined behavior has no guarantees at all; implementation-defined behavior is guaranteed but may vary between compilers (e.g., the size of an `int`).

Overflow — When a calculation produces a value outside the range a type can represent (e.g., adding 1 to the maximum `int` value).

```
        int maxInt = 2147483647;
        int overflowed = maxInt + 1; // undefined behavior for signed
integers
```

Type safety — A language property ensuring that operations are only performed on compatible data types, catching many errors at compile time.

Undefined symbol / linker error — An error that occurs when the linker cannot find the implementation of a declared function or variable.

Header file vs. source file — A header (`.h`/`.hpp`) typically contains declarations; a source file (`.cpp`) contains definitions/implementations.

Compilation flags — Options passed to the compiler to control optimization, warnings, and standard version (e.g., `-std=c++20 -Wall -O2`).

Undefined initialization — A variable declared without an initializer may contain garbage (indeterminate) values until assigned.

```
        int x;                  // uninitialized: value is indeterminate
```

```
        std::cout << x;         // undefined behavior — could print anything
```

Idiom — A common, established pattern for solving a recurring problem in a language (e.g., RAII, PIMPL, CRTP).

PIMPL (Pointer to Implementation) — A technique that hides a class's implementation details behind a pointer to reduce compilation dependencies.

CRTP (Curiously Recurring Template Pattern) — A technique where a class inherits from a template instantiation of itself, used for static polymorphism.

```
        template <typename Derived>
        class Base {
        public:
            void interface() { static_cast<Derived*>(this)-
>implementation(); }
        };
        class Derived : public Base<Derived> {
        public:
            void implementation() { std::cout << "Derived implementation"; }
        };
```

---

35. Conclusion

This report has walked through the C++ language from its most foundational building blocks — variables, operators, and control flow — through its object-oriented core of classes, inheritance, and polymorphism, into its more advanced facilities such as templates, the STL, smart pointers, lambdas, and concurrency, finishing with the modern features introduced from C++11 through C++23.

Each term was paired with a working code example to ground the concept in practice rather than abstract definition alone. Together, these building blocks explain why C++ continues to power operating systems, browsers, game engines, and performance-critical software: it offers a rare combination of low-level control and high-level expressive power, backed by a standard that continues to evolve while preserving backward compatibility.

Mastery of C++ comes from combining this vocabulary with deliberate practice — writing, compiling, and debugging real programs — since many of its subtleties (memory management, undefined behavior, template mechanics) are best understood by encountering them directly.

End of Report